

Versuchsprotokoll Versuch 6 - RISC-V

Praktikum: Digitaltechnik

Versuch: 6 - RISC-V

Name: Precious Adaugo Nnamaka

Betreuer: Prof. Dr.-Ing. Peter Zipf

1 Ziel des Versuchs

Ziel des sechsten Versuchs war die Entwicklung zentraler Funktionsblöcke einer 32-Bit-RISC-V-CPU nach dem RV32I-Befehlssatz. Bearbeitet wurden der Befehlsdecoder, eine ALU mit Barrel-Shiftern sowie die Logik des Programmzählers. Die Schaltungen wurden zunächst modular in Digital aufgebaut und simuliert. Anschließend wurden die Entwürfe nach VHDL exportiert und in Vivado auf einem PYNQ-Z1-Board mit dem FPGA XC7Z020 umgesetzt.

Der Versuch zeigte damit den Übergang von einzelnen digitalen Funktionsblöcken zu Bestandteilen eines einfachen Prozessordatenpfades.

2 Verwendete Werkzeuge und Dateien

Werkzeug / Datei | Verwendung

Digital | Grafischer Entwurf und Simulation der Schaltungen

Vivado | Synthese, Implementation, Bitstream-Erzeugung und Programmierung

PYNQ-Z1 / XC7Z020 | FPGA-Zielhardware

v6_decoder.dig | Entwurf und Simulation des Befehlsdecoders

v6_decoder.vhdl | Exportierte VHDL-Beschreibung des Decoders

v6_right_shifter_test_2.dig | Test des selbst entwickelten Rechtsshifters

v6_alu.dig | Entwurf der ALU

v6_program_counter.dig | Entwurf der Programmzählerlogik

v6_program_counter.vhdl | Exportierte VHDL-Beschreibung des Programmzählers

3 Theoretische Grundlagen

RISC-V ist eine offene Prozessorarchitektur mit regelmäßig aufgebauten Befehlsformaten. Im hier betrachteten RV32I-Befehlssatz sind die Befehle 32 Bit breit. Ein Prozessor muss aus jedem Befehl die benötigten Felder auslesen, die passende Operation bestimmen und anschließend den nächsten Wert des Programmzählers berechnen.

Ein vereinfachter Datenpfad besteht aus folgenden Hauptblöcken:

Block | Aufgabe | Wichtige Schnittstellen

Befehlsdecoder | Auslesen und Klassifizieren der Instruktion | $\text{instr}_i[31:0] \rightarrow \text{opcode}, \text{funct3}, \text{funct7}, \text{rs1}, \text{rs2}, \text{rd}, \text{type}$

ALU | Arithmetische, logische und Vergleichsoperationen | $\text{a}_i, \text{b}_i, \text{op}_i \rightarrow \text{s}_o$

Barrel-Shifter | Schnelle Schiebeoperationen um 0 bis 31 Stellen | Eingangswert, Schiebeweite, Modus

Programmzähler | Berechnung der nächsten Befehlsadresse | $\text{pc}_i, \text{imm}_i, \text{a}_i, \text{b}_i, \text{jmpcond}_i \rightarrow \text{pc}_o$

Im normalen sequentiellen Ablauf wird der Programmzähler um 4 erhöht, da jeder Befehl 32 Bit beziehungsweise 4 Byte lang ist:

$$\text{pc}_o = \text{pc}_i + 4$$

Bei Sprung- oder Branch-Befehlen kann der nächste Programmzählerwert stattdessen aus einer Immediate-Adresse oder aus einer registerrelativen Adresse berechnet werden.

4 Befehlsdecoder

Der Befehlsdecoder zerlegt eine 32-Bit-Instruktion in ihre einzelnen Felder. Dazu wurden in Digital Splitter verwendet. Der Decoder gibt opcode_o , funct3_o und funct7_o direkt aus. Die Registeradressen rs1_o , rs2_o und rd_o werden je nach Befehlsformat aus den passenden Bitbereichen ausgelesen. Falls ein Feld in einem bestimmten Instruktionsformat nicht vorhanden ist, wird als Default-Wert 0 ausgegeben.

Zusätzlich erzeugt der Decoder das Signal type_o . Dieses Signal beschreibt das Instruktionsformat:

type_o | Format

000 | R-Typ

001 | I-Typ

010 | S-Typ

011 | B-Typ

100 | U-Typ

101 | J-Typ

Die wichtigsten Ausgaenge des Decoders sind:

Ausgang | Bitbreite | Funktion

opcode_o | 7 Bit | Operationscode der Instruktion

funct3_o | 3 Bit | Unterfunktion

funct7_o | 7 Bit | Erweiterte Unterfunktion

rs1_o | 5 Bit | Adresse des ersten Quellregisters

rs2_o | 5 Bit | Adresse des zweiten Quellregisters

rd_o | 5 Bit | Adresse des Zielregisters

type_o | 3 Bit | Codierung des Instruktionsformats

Der Decoder wurde in Digital mit Beispielbefehlen der Formate R, I, S, B, U und J simuliert. Die Ausgaenge entsprachen den erwarteten Bitfeldern und der vorgegebenen Typcodierung. Anschliessend wurde der Wrapper nach VHDL exportiert. Dadurch wurden auch die benoetigten Unterschaltungen in die Datei v6_decoder.vhdl aufgenommen.

5 Barrel-Shifter

Fuer die ALU wurden Schiebeoperationen benoetigt. Ein Barrel-Shifter kann einen Wert in einem einzigen kombinatorischen Schaltnetz um mehrere Stellen verschieben. Der 32-Bit-Barrel-Shifter wurde aus fuenf Stufen aufgebaut. Diese Stufen verschieben bei Bedarf um 1, 2, 4, 8 oder 16 Bit. Durch die Kombination dieser Stufen lassen sich alle Schiebeweiten von 0 bis 31 darstellen.

Es wurden drei Varianten betrachtet:

Operation | Bedeutung

sll | Logischer Linksshift, rechts werden Nullen eingefuegt

srl | Logischer Rechtsshift, links werden Nullen eingefuegt

sra | Arithmetischer Rechtsshift, links wird das Vorzeichenbit aufgefuellt

Beim arithmetischen Rechtsshift ist entscheidend, dass das hoechstwertige Bit als Vorzeichenbit erhalten bleibt. Dadurch bleiben negative Zahlen in Zweierkomplementdarstellung auch nach dem Schieben negativ.

Der selbst entwickelte Rechtsshifter wurde in v6_right_shifter_test_2.dig mit dem integrierten Digital-Barrel-Shifter verglichen. Als Testwerte dienten unter anderem 0xDEADBEEF und dessen bitweise Inversion. Alle Schiebeweiten von 0 bis 31 wurden fuer logisches und arithmetisches Schieben geprueft. Bei einer Abweichung haette der Vergleichsausgang einen Fehler angezeigt. Der Test lief ohne festgestellte Abweichung durch.

6 ALU

Die ALU berechnet arithmetische, logische und vergleichende Operationen. In v6_alu.dig wurden die einzelnen Funktionsbloecke parallel aufgebaut. Ein Multiplexer waehlt anschliessend anhand des Steuersignals op_i das gueltige Ergebnis aus.

Die implementierten Operationen waren:

op_i | Operation

0000 | add

0001 | sub

0010 | xor

0011 | or

0100 | and

0101 | sll

0110 | srl

0111 | sra

1000 | slt

1001 | sltu

Die Operation slt fuehrt einen vorzeichenbehafteten Vergleich durch. Die Operation sltu fuehrt einen vorzeichenlosen Vergleich durch. Dadurch muss die ALU zwischen signed und unsigned Auswertung unterscheiden koennen. Die selbst entwickelten Shift-Schaltungen wurden als Unterschaltungen in die ALU eingebunden.

7 Programmzaehler

Der Programmzaehler bestimmt die Adresse des naechsten auszufuehrenden Befehls. Fuer den normalen Programmablauf gilt:

$pc_o = pc_i + 4$

Bei Branch- und Sprungbefehlen muss dagegen ein anderer Folgewert ausgewaehlt werden. Die Auswahl erfolgt ueber das Signal jmpcond_i.

jmpcond_i | Bedeutung | Auswahl

0000 | normaler Ablauf | $pc_i + 4$

0001 | beq | Sprung bei Gleichheit

0010 | bne | Sprung bei Ungleichheit

0011 | blt | signed kleiner als

0100 | bge | signed groesser/gleich

0101 | bltu | unsigned kleiner als

0110 | bgeu | unsigned groesser/gleich

0111 | jal | $pc_i + \text{Immediate}$

1000 | jalr | $a_i + \text{Immediate}$

Die Programmzaehlerlogik enthaelt Addierer, Vergleichslogik und Multiplexer. Bei bedingten Spruengen wird anhand von a_i und b_i entschieden, ob $pc_i + \text{imm}_i$ oder $pc_i + 4$ ausgegeben wird. Bei jal wird direkt $pc_i + \text{imm}_i$ verwendet. Bei jalr wird die neue Adresse aus $a_i + \text{imm}_i$ gebildet.

Die Schaltung v6_program_counter.dig wurde fuer den normalen Ablauf, fuer alle Branch-Bedingungen sowie fuer jal und jalr simuliert. Der Wrapper wurde anschliessend als v6_program_counter.vhdl exportiert.

8 Gemeinsame FPGA-Durchfuehrung

Abweichend von einer getrennten Umsetzung der einzelnen Teilaufgaben wurden Befehlsdecoder, ALU einschliesslich der Shift-Schaltungen und Programmzaehler in das uebergeordnete RISC-V-VHDL-Gesamtdesign eingebunden. Dadurch musste Synthese, Implementation,

Bitstream-Erzeugung und Hardwaretest nur einmal fuer das vollstaendige Design ausgefuehrt werden. Vor der Programmierung wurden pmod_a_0 und pmod_a_1 mit einem Kabel verbunden. Diese Rueckkopplung war notwendig, damit das Testsystem die vorgesehene UART-Kommunikation nutzen konnte.

Nach der Programmierung fuehrte das Gesamtdesign die integrierten Tests aus. Die Statusanzeige des Boards zeigte den Testzustand an:

Anzeige | Bedeutung

gruen / LD4 | Test erfolgreich abgeschlossen

rot | Fehler erkannt

blau oder violett | Test laeuft noch

Bei der Durchfuehrung signalisierte die grueene Anzeige LD4 einen erfolgreichen Abschluss. Damit wurde bestaetigt, dass die eingebundenen Hauptbloecke im Gesamtdesign korrekt zusammenarbeiteten.

9 Auswertung

Die modulare Vorgehensweise erleichterte die Entwicklung und Fehlersuche. Decoder, Shifter, ALU und Programmzaehler konnten zunaechst einzeln in Digital geprueft werden. Anschliessend wurden sie im VHDL-Gesamtdesign zusammengefuehrt.

Der Befehlsdecoder stellte die benoetigten Steuer- und Registerinformationen bereit. Die ALU berechnete Datenwerte, logische Operationen, Schiebeoperationen und Vergleiche. Der Programmzaehler setzte daraus den Kontrollfluss um, indem er zwischen sequentielltem Ablauf, bedingten Spruengen und direkten Spruengen unterschied.

Der erfolgreiche FPGA-Test zeigte, dass die drei Hauptbloecke innerhalb des RISC-V-Gesamtdesigns korrekt zusammenwirkten. Dadurch war eine wiederholte einzelne Programmierung der Teilprojekte nicht notwendig.

10 Fehlerbetrachtung

Moegliche Fehlerquellen bei diesem Versuch waren:

1. Falsch zugeordnete Bitfelder im Decoder koennen zu falschen Registeradressen oder falscher Typcodierung fuehren.
2. Eine Verwechslung der RISC-V-Formate R, I, S, B, U und J kann die Immediate- oder Registerauswertung verfälschen.
3. Beim arithmetischen Rechtsshift muss das Vorzeichenbit korrekt aufgefuellt werden.
4. Signed- und unsigned-Vergleiche duerfen bei slt, sltu sowie bei Branch-Bedingungen nicht verwechselt werden.
5. Eine falsche Auswahl im Multiplexer der ALU kann trotz korrekt berechneter Teiloperation ein falsches Ergebnis liefern.
6. Beim Programmzaehler koennen Fehler in der Branch-Bedingung zu falschen Sprungadressen fuehren.
7. Beim VHDL-Export muessen Wrapper und Unterschaltungen vollstaendig eingebunden sein.
8. Eine fehlende Rueckkopplung zwischen pmod_a_0 und pmod_a_1 kann den Hardwaretest verhindern.

11 Fazit

Der Versuch zeigte, wie zentrale Bestandteile einer einfachen RISC-V-CPU entworfen, simuliert und auf einem FPGA getestet werden koennen. Der Decoder zerlegte 32-Bit-Instruktionen in ihre relevanten Felder und klassifizierte das Instruktionsformat. Die ALU stellte arithmetische, logische, Schiebe- und Vergleichsoperationen bereit. Die Programmzaehlerlogik bestimmte den naechsten Befehlswert fuer normalen Ablauf, Branches und Spruenge.

Durch die gemeinsame Einbindung in das RISC-V-Gesamtdesign konnte die Funktion der Teilbloecke im Zusammenspiel getestet werden. Die grueene Statusanzeige LD4 bestaetigte den erfolgreichen Hardwaretest. Insgesamt verdeutlichte der Versuch den Zusammenhang zwischen Digital-Entwurf, VHDL-Beschreibung, FPGA-Umsetzung und Prozessorarchitektur.